```
# Experiment 7: Customer Segmentation using K-Means Clustering

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.preprocessing import StandardScaler
from sklearn.cluster import KMeans
from sklearn.metrics import silhouette_score
from sklearn.decomposition import PCA

import warnings
warnings.filterwarnings("ignore")

print("Libraries imported successfully!")
```

```
Libraries imported successfully!
```

```
# Load Mall Customer dataset

url = "https://raw.githubusercontent.com/sharmaroshan/Clustering-of-Mall-Customers/master/Mall_Customers.csv"

df = pd.read_csv(url)

print("Dataset loaded successfully!")
print("Shape of dataset:", df.shape)

df.head()
```

```
Dataset loaded successfully!
Shape of dataset: (200, 5)
```

	CustomerID	Genre	Age	Annual Income (k$)	Spending Score (1-100)
0	1	Male	19	15	39
1	2	Male	21	15	81
2	3	Female	20	16	6
3	4	Female	23	16	77
4	5	Female	31	17	40

Next steps: Generate code with `df` New interactive sheet

```
# Display basic information

print("Dataset Information:\n")
df.info()

print("\n\nMissing Values:")
print(df.isnull().sum())

print("\n\nDuplicate Rows:")
print(df.duplicated().sum())
```

```
Dataset Information:

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 200 entries, 0 to 199
Data columns (total 5 columns):
 #   Column                  Non-Null Count  Dtype
---  ------                  --------------  -----
 0   CustomerID              200 non-null    int64
 1   Genre                   200 non-null    object
 2   Age                     200 non-null    int64
 3   Annual Income (k$)      200 non-null    int64
 4   Spending Score (1-100)  200 non-null    int64
dtypes: int64(4), object(1)
memory usage: 7.9+ KB


Missing Values:
CustomerID                0
Genre                     0
Age                       0
Annual Income (k$)        0
Spending Score (1-100)    0
dtype: int64
```

```
Duplicate Rows:
0
```

```
# Statistical summary of numerical columns

df.describe()
```

	CustomerID	Age	Annual Income (k$)	Spending Score (1-100)
count	200.000000	200.000000	200.000000	200.000000
mean	100.500000	38.850000	60.560000	50.200000
std	57.879185	13.969007	26.264721	25.823522
min	1.000000	18.000000	15.000000	1.000000
25%	50.750000	28.750000	41.500000	34.750000
50%	100.500000	36.000000	61.500000	50.000000
75%	150.250000	49.000000	78.000000	73.000000
max	200.000000	70.000000	137.000000	99.000000

```
# Remove duplicate rows
df = df.drop_duplicates()

# Handle missing numerical values using median
numeric_columns = df.select_dtypes(include=np.number).columns

for col in numeric_columns:
    df[col] = df[col].fillna(df[col].median())

print("Missing values after preprocessing:")
print(df.isnull().sum())

print("\nDataset shape after preprocessing:", df.shape)
```

```
Missing values after preprocessing:
CustomerID                0
Genre                     0
Age                       0
Annual Income (k$)        0
Spending Score (1-100)    0
dtype: int64

Dataset shape after preprocessing: (200, 5)
```

```
# Select features for clustering

features = [
    'Age',
    'Annual Income (k$)',
    'Spending Score (1-100)'
]

X = df[features]

print("Selected features:")
print(X.head())

print("\nFeature shape:", X.shape)
```

```
Selected features:
   Age  Annual Income (k$)  Spending Score (1-100)
0   19                  15                      39
1   21                  15                      81
2   20                  16                       6
3   23                  16                      77
4   31                  17                      40

Feature shape: (200, 3)
```

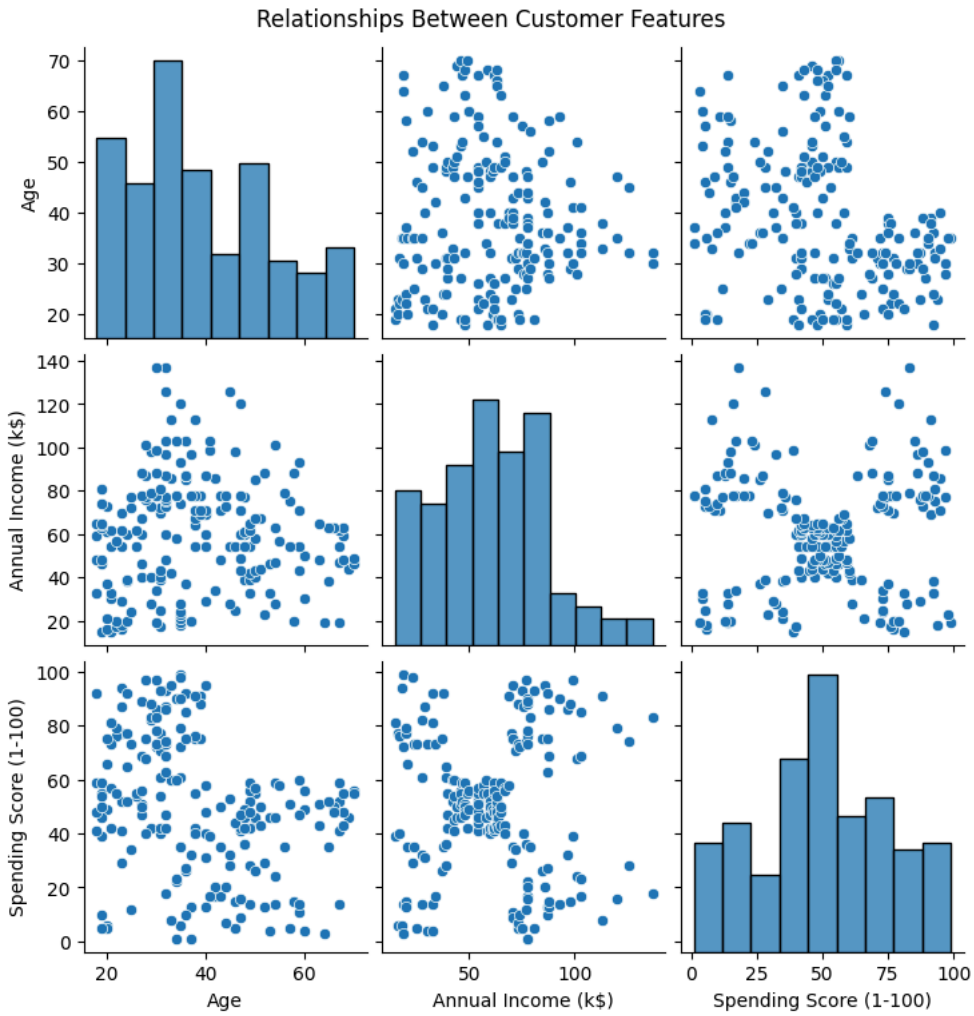
```
# Pairwise visualization of selected features

sns.pairplot(
    df[features],
    diag_kind="hist"
)

plt.suptitle(
    "Relationships Between Customer Features",
    y=1.02
```

```
)

plt.show()
```




```
# Standardize the features

scaler = StandardScaler()

X_scaled = scaler.fit_transform(X)

X_scaled = pd.DataFrame(
    X_scaled,
    columns=features
)

print("Scaled data:")
print(X_scaled.head())
```

```
Scaled data:
        Age  Annual Income (k$)  Spending Score (1-100)
0 -1.424569           -1.738999               -0.434801
1 -1.281035           -1.738999                1.195704
2 -1.352802           -1.700830               -1.715913
3 -1.137502           -1.700830                1.040418
4 -0.563369           -1.662660               -0.395980
```

```
# Elbow Method

inertia = []

K_range = range(2, 11)

for k in K_range:

    kmeans = KMeans(
        n_clusters=k,
        random_state=42,
        n_init=10
    )
```

```
    kmeans.fit(X_scaled)

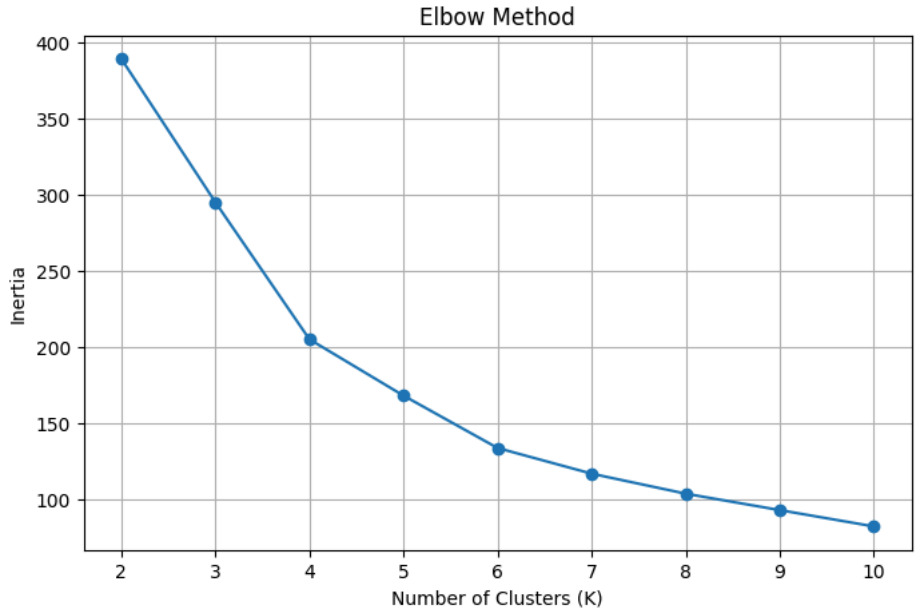
    inertia.append(kmeans.inertia_)

plt.figure(figsize=(8, 5))

plt.plot(
    K_range,
    inertia,
    marker='o'
)

plt.title("Elbow Method")
plt.xlabel("Number of Clusters (K)")
plt.ylabel("Inertia")
plt.xticks(K_range)
plt.grid(True)

plt.show()
```




```
# Calculate silhouette score for each K

silhouette_scores = []

for k in K_range:

    kmeans = KMeans(
        n_clusters=k,
        random_state=42,
        n_init=10
    )

    labels = kmeans.fit_predict(X_scaled)

    score = silhouette_score(
        X_scaled,
        labels
    )

    silhouette_scores.append(score)

# Display scores

silhouette_results = pd.DataFrame({
    "K": list(K_range),
    "Silhouette Score": silhouette_scores
})

print(silhouette_results)

# Plot

plt.figure(figsize=(8, 5))

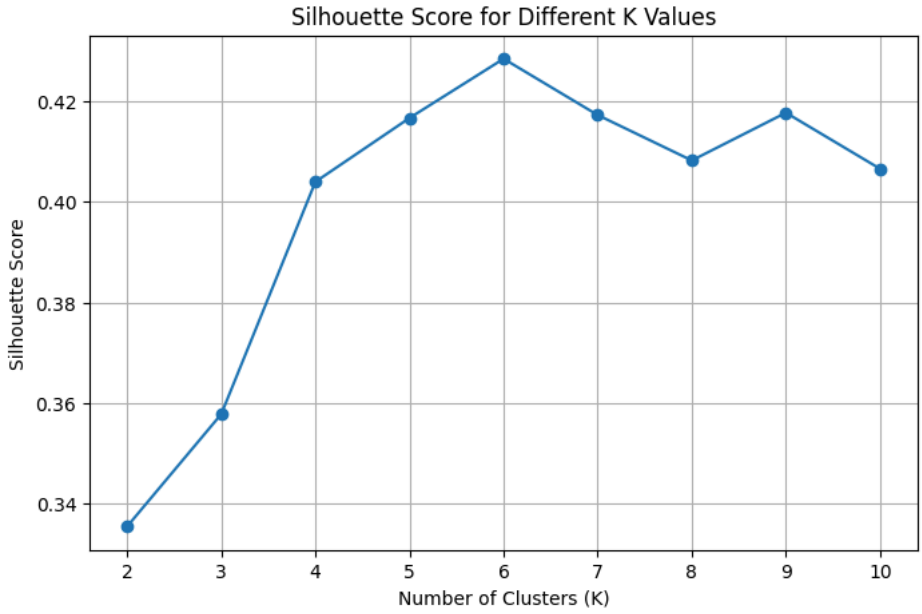
plt.plot(
    list(K_range),
```

```
    silhouette_scores,
    marker='o'
)

plt.title("Silhouette Score for Different K Values")
plt.xlabel("Number of Clusters (K)")
plt.ylabel("Silhouette Score")
plt.xticks(list(K_range))
plt.grid(True)

plt.show()
```

```
    K  Silhouette Score
0   2          0.335472
1   3          0.357793
2   4          0.403958
3   5          0.416643
4   6          0.428417
5   7          0.417232
6   8          0.408207
7   9          0.417693
8  10          0.406554
```




```
# Select K with the highest silhouette score

best_k = list(K_range)[
    np.argmax(silhouette_scores)
]

best_score = max(silhouette_scores)

print("Best K based on Silhouette Score:", best_k)
print("Best Silhouette Score:", round(best_score, 4))
```

```
Best K based on Silhouette Score: 6
Best Silhouette Score: 0.4284
```

```
# Apply K-Means using selected K

kmeans = KMeans(
    n_clusters=best_k,
    random_state=42,
    n_init=10
)

df["Cluster"] = kmeans.fit_predict(X_scaled)

print("K-Means clustering completed successfully!")

print("\nCluster distribution:")
print(df["Cluster"].value_counts().sort_index())
```

```
K-Means clustering completed successfully!

Cluster distribution:
Cluster
0    45
1    39
```

```
2    33
3    39
4    23
5    21
Name: count, dtype: int64
```

```
# Convert scaled centroids back to original feature units

centroids_scaled = kmeans.cluster_centers_

centroids_original = scaler.inverse_transform(
    centroids_scaled
)

centroids = pd.DataFrame(
    centroids_original,
    columns=features
)

centroids["Cluster"] = range(best_k)

print("Cluster Centroids:")
print(centroids)
```

```
Cluster Centroids:
         Age  Annual Income (k$)  Spending Score (1-100)  Cluster
0  56.333333           54.266667               49.066667        0
1  26.794872           57.102564               48.128205        1
2  41.939394           88.939394               16.969697        2
3  32.692308           86.538462               82.128205        3
4  25.000000           25.260870               77.608696        4
5  45.523810           26.285714               19.380952        5
```

```
# Reduce 3 dimensions to 2 dimensions using PCA

pca = PCA(n_components=2)

X_pca = pca.fit_transform(X_scaled)

df["PCA1"] = X_pca[:, 0]
df["PCA2"] = X_pca[:, 1]

print(
    "Variance explained by PCA components:",
    pca.explained_variance_ratio_
)
```

```
Variance explained by PCA components: [0.44266167 0.33308378]
```

```
# Visualize customer clusters

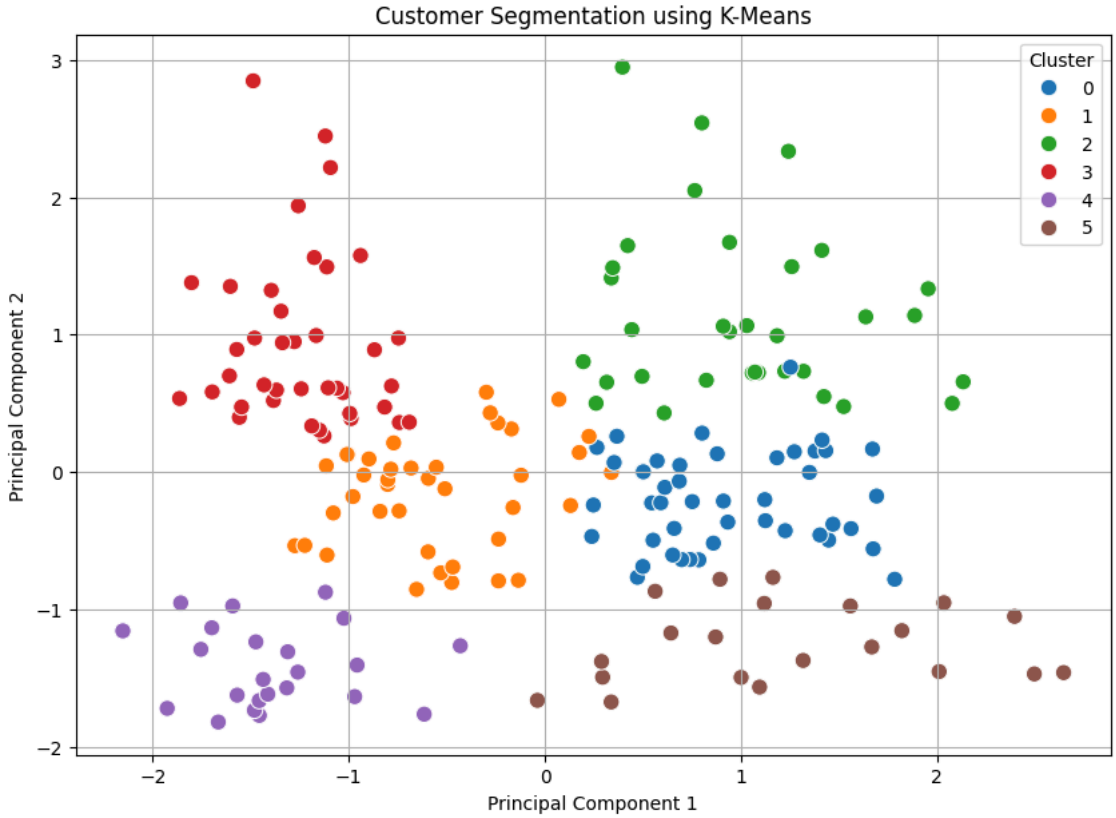
plt.figure(figsize=(10, 7))

sns.scatterplot(
    data=df,
    x="PCA1",
    y="PCA2",
    hue="Cluster",
    palette="tab10",
    s=80
)

plt.title("Customer Segmentation using K-Means")
plt.xlabel("Principal Component 1")
plt.ylabel("Principal Component 2")

plt.legend(title="Cluster")

plt.grid(True)
plt.show()
```

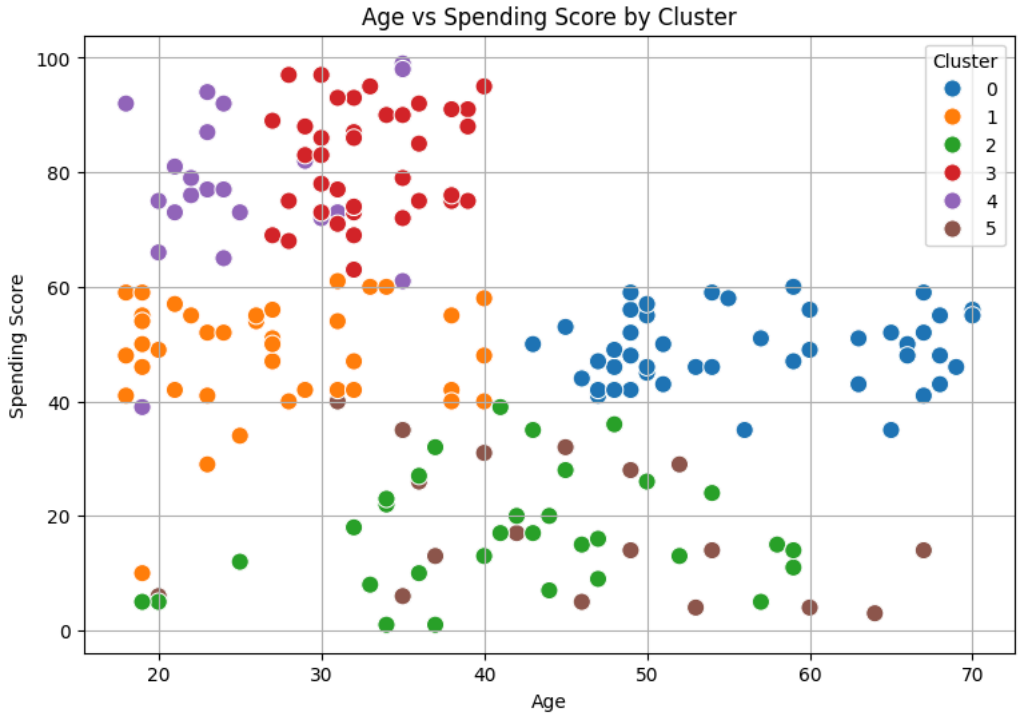



```
plt.figure(figsize=(9, 6))

sns.scatterplot(
    data=df,
    x="Age",
    y="Spending Score (1-100)",
    hue="Cluster",
    palette="tab10",
    s=90
)

plt.title("Age vs Spending Score by Cluster")
plt.xlabel("Age")
plt.ylabel("Spending Score")
plt.grid(True)

plt.show()
```

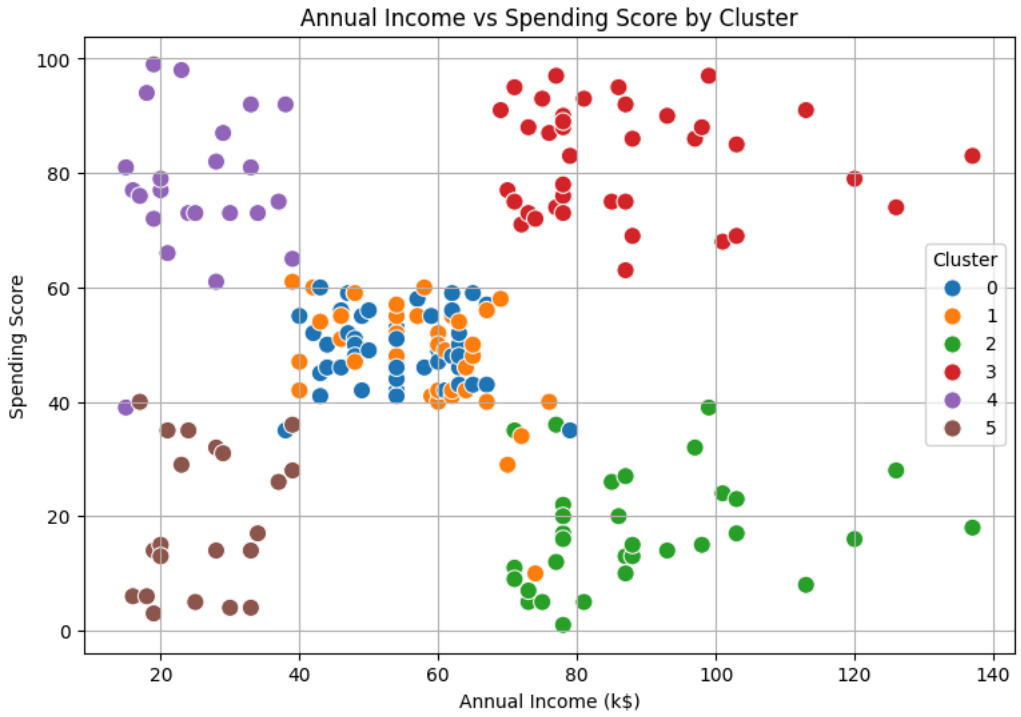


```
plt.figure(figsize=(9, 6))

sns.scatterplot(
    data=df,
    x="Annual Income (k$)",
    y="Spending Score (1-100)",
    hue="Cluster",
    palette="tab10",
    s=90
)

plt.title("Annual Income vs Spending Score by Cluster")
plt.xlabel("Annual Income (k$)")
plt.ylabel("Spending Score")
plt.grid(True)

plt.show()
```




```
# Calculate average characteristics of each cluster

cluster_summary = df.groupby("Cluster")[features].mean()

cluster_summary["Customer Count"] = (
    df["Cluster"].value_counts()
)

cluster_summary = cluster_summary.sort_index()

print("Customer Segment Summary:")
display(cluster_summary.round(2))
```

Customer Segment Summary:

Cluster	Age	Annual Income (k$)	Spending Score (1-100)	Customer Count
0	56.33	54.27	49.07	45
1	26.79	57.10	48.13	39
2	41.94	88.94	16.97	33
3	32.69	86.54	82.13	39
4	25.00	25.26	77.61	23
5	45.52	26.29	19.38	21

```
# Print cluster characteristics

for cluster in sorted(df["Cluster"].unique()):
```

```
    cluster_data = df[df["Cluster"] == cluster]

    avg_age = cluster_data["Age"].mean()
    avg_income = cluster_data["Annual Income (k$)"].mean()
    avg_spending = cluster_data["Spending Score (1-100)"].mean()
    count = len(cluster_data)

    print(f"\nCluster {cluster}")
    print("-" * 40)
    print(f"Customers       : {count}")
    print(f"Average Age     : {avg_age:.2f}")
    print(f"Average Income  : {avg_income:.2f} k$")
    print(f"Avg Spending    : {avg_spending:.2f}")
```

```
Cluster 0
----------------------------------------
Customers       : 45
Average Age     : 56.33
Average Income  : 54.27 k$
Avg Spending    : 49.07

Cluster 1
----------------------------------------
Customers       : 39
Average Age     : 26.79
Average Income  : 57.10 k$
Avg Spending    : 48.13

Cluster 2
----------------------------------------
Customers       : 33
Average Age     : 41.94
Average Income  : 88.94 k$
Avg Spending    : 16.97

Cluster 3
----------------------------------------
Customers       : 39
Average Age     : 32.69
Average Income  : 86.54 k$
Avg Spending    : 82.13

Cluster 4
----------------------------------------
Customers       : 23
Average Age     : 25.00
Average Income  : 25.26 k$
Avg Spending    : 77.61

Cluster 5
----------------------------------------
Customers       : 21
Average Age     : 45.52
Average Income  : 26.29 k$
Avg Spending    : 19.38
```

```
# Create simple customer segment descriptions

summary = df.groupby("Cluster")[features].mean()

for cluster in summary.index:

    income = summary.loc[cluster, "Annual Income (k$)"]
    spending = summary.loc[cluster, "Spending Score (1-100)"]

    if income < df["Annual Income (k$)"].median() and spending < df["Spending Score (1-100)"].median():
        segment = "Low Income - Low Spending"

    elif income < df["Annual Income (k$)"].median() and spending >= df["Spending Score (1-100)"].median():
        segment = "Low Income - High Spending"

    elif income >= df["Annual Income (k$)"].median() and spending < df["Spending Score (1-100)"].median():
        segment = "High Income - Low Spending"

    else:
        segment = "High Income - High Spending"

    print(f"Cluster {cluster}: {segment}")
```

```
Cluster 0: Low Income - Low Spending
Cluster 1: Low Income - Low Spending
Cluster 2: High Income - Low Spending
Cluster 3: High Income - High Spending
Cluster 4: Low Income - High Spending
Cluster 5: Low Income - Low Spending
```

```
print("Suggested Marketing Strategies\n")

for cluster in sorted(df["Cluster"].unique()):

    cluster_data = df[df["Cluster"] == cluster]

    avg_income = cluster_data["Annual Income (k$)"].mean()
    avg_spending = cluster_data["Spending Score (1-100)"].mean()

    print(f"\nCluster {cluster}")
    print("-" * 40)

    if avg_income >= df["Annual Income (k$)"].median() and avg_spending >= df["Spending Score (1-100)"].median():
        print("Strategy: Premium products, loyalty programs and personalized offers.")

    elif avg_income < df["Annual Income (k$)"].median() and avg_spending >= df["Spending Score (1-100)"].median():
        print("Strategy: Discounts, attractive bundles and promotional campaigns.")

    elif avg_income >= df["Annual Income (k$)"].median() and avg_spending < df["Spending Score (1-100)"].median():
        print("Strategy: Personalized recommendations and campaigns to increase engagement.")

    else:
        print("Strategy: Budget-friendly products and awareness campaigns.")
```

```
Suggested Marketing Strategies


Cluster 0
----------------------------------------
Strategy: Budget-friendly products and awareness campaigns.

Cluster 1
----------------------------------------
Strategy: Budget-friendly products and awareness campaigns.

Cluster 2
----------------------------------------
Strategy: Personalized recommendations and campaigns to increase engagement.

Cluster 3
----------------------------------------
Strategy: Premium products, loyalty programs and personalized offers.

Cluster 4
----------------------------------------
Strategy: Discounts, attractive bundles and promotional campaigns.

Cluster 5
----------------------------------------
Strategy: Budget-friendly products and awareness campaigns.
```

```
print("""
CONCLUSION

K-Means clustering was successfully applied to the Mall Customer dataset.
The Elbow Method and Silhouette Score were used to determine a suitable
number of customer clusters.

Customers were grouped according to age, annual income and spending score.
The resulting clusters represent different customer behaviors and can help
businesses design targeted marketing campaigns, improve customer engagement,
personalize offers and improve customer retention.
""")
```

```
CONCLUSION

K-Means clustering was successfully applied to the Mall Customer dataset.
The Elbow Method and Silhouette Score were used to determine a suitable
number of customer clusters.

Customers were grouped according to age, annual income and spending score.
The resulting clusters represent different customer behaviors and can help
businesses design targeted marketing campaigns, improve customer engagement,
personalize offers and improve customer retention.
```

Ques 1: What is clustering? How does it differ from classification?

Ans: Clustering is an unsupervised machine learning technique used to divide data into groups called clusters based on the similarity between data points. It does not require predefined labels. Classification is a supervised machine learning technique in which the

model learns from labeled data and assigns new data to predefined classes. For example, customer segmentation is a clustering problem, while predicting whether an email is spam or not spam is a classification problem.

Ques 2: Explain the working principle of the K-Means Clustering algorithm.

Ans: K-Means is an unsupervised clustering algorithm that divides data into a specified number of clusters, represented by K. First, K centroids are initialized. Each data point is then assigned to the nearest centroid based on distance. After that, new centroids are calculated using the mean of the points belonging to each cluster. This process of assigning points and recalculating centroids is repeated until the centroids become stable and the clusters no longer change significantly.

Ques 3: Why is feature scaling important before applying K-Means clustering?

Ans: Feature scaling is important because K-Means uses distance calculations to assign data points to clusters. If different features have different numerical ranges, a feature with larger values can have a greater influence on the clustering result. Standardization or normalization brings the features to a comparable scale so that each feature contributes fairly to the distance calculation. Therefore, feature scaling helps produce more meaningful clustering results.

Ques 4: What is the Elbow Method, and how does it help determine the optimal number of clusters?

Ans: The Elbow Method is a technique used to determine a suitable number of clusters for K-Means. It involves calculating the within-cluster sum of squared distances, called inertia, for different values of K and plotting them on a graph. As the value of K increases, the inertia decreases. The point where the decrease starts becoming slower forms an elbow in the graph, and this point is generally selected as the suitable number of clusters.

Ques 5: What is the Silhouette Score? How is it used to evaluate clustering performance?

Ans: The Silhouette Score is a metric used to evaluate the quality of clustering. It measures how similar a data point is to the points in its own cluster compared with points in other clusters. The score generally ranges from -1 to +1. A value closer to +1 indicates that the clusters are well separated and the data points are appropriately assigned. The Silhouette Score can be calculated for different values of K to help select a suitable clustering solution.

Ques 6: Why is K-Means considered an unsupervised machine learning algorithm?

Ans: K-Means is considered an unsupervised machine learning algorithm because it works with data that does not have predefined output labels or classes. The algorithm automatically identifies groups or patterns within the data based on the similarity of the observations. In customer segmentation, customers are grouped according to characteristics such as age, income, and spending behavior without being given predefined customer categories.

Ques 7: Mention any four real-world applications of customer segmentation.

Ans: Customer segmentation can be used in many real-world applications. Four important applications are targeted marketing campaigns, personalized product recommendations, improving customer engagement, and customer retention strategies. Businesses can also use customer segmentation to understand customer purchasing behavior and improve their overall business performance.

Ques 8: What are the limitations of the K-Means algorithm?

Ans: K-Means has several limitations. The number of clusters, K, generally needs to be selected before applying the algorithm. The algorithm can also be sensitive to the initial placement of centroids, and outliers can affect the resulting clusters. K-Means works best when clusters are relatively well separated and suitable for distance-based grouping. It may not perform well when the data contains complex or irregularly shaped clusters.

Ques 9: How can businesses use customer segmentation to improve marketing and customer retention?

Ans: Businesses can use customer segmentation to divide customers into groups based on their characteristics, spending behavior, and purchasing patterns. Each group can then receive a suitable marketing strategy. For example, high-value customers can receive premium offers and loyalty rewards, while customers with low spending can be targeted with discounts and promotional campaigns. By providing relevant offers and communication to different customer groups, businesses can improve customer engagement, satisfaction, and retention.

Ques 10: Compare K-Means Clustering with Hierarchical Clustering based on their working principles and applications.

Ans: K-Means Clustering divides data into a predefined number of clusters by repeatedly assigning data points to the nearest centroid and recalculating the centroids. Hierarchical Clustering creates a hierarchy of clusters by progressively merging similar groups or dividing larger groups into smaller ones. K-Means is generally suitable when the required number of clusters is known or can be estimated and is commonly used for customer segmentation. Hierarchical Clustering is useful when the relationship between groups is important and can be represented using a dendrogram.